

RWA Tokenization Platform

A reserve-backed asset tokenization protocol with on-chain governance, ERC-4626 vaults, constant-product AMM, UUPS-upgradeable registry, and Chainlink-compatible oracle integration. Deployed on Base Sepolia

PROJECT

Blockchain Technologies 2 – Option C

COMMIT HASH

8ac3c73

SOLIDITY VERSION

0.8.20 · via_ir · optimizer 200 runs

STATUS

Final — for submission

REPOSITORY

Blockchain2_FINAL_Alikhan_Zarina

DOCUMENT TYPE

Architecture · C4 + ADR

TARGET CHAIN

Base Sepolia

AUTHORS

Alikhan · Zarina

Table of Contents

1.	Executive Summary	3
2.	System Overview.....	6
3.	System Context (C4 Level 1).....	8
4.	Container & Component Architecture.....	10
5.	Access Control Model	13
6.	Governance Lifecycle.....	17
7.	AMM Swap Flow.....	19
8.	Vault Deposit & Redeem Flow.....	21
9.	Data Model & Storage Layout	23
10.	Upgradeability Architecture	25
11.	Oracle Architecture	27
12.	The Graph / Indexing Architecture.....	28
13.	CI/CD & DevSecOps Architecture.....	30
14.	Architecture Decision Records (ADR).....	32
15.	Scalability & Future Improvements.....	35
16.	Conclusion.....	36

1. Executive Summary

Architecture of Option C: Real World Asset Tokenization platform is presented within this paper and is implemented using Solidity 0.8.20 version with React/Wagmi front end, and The Graph subgraph. Protocol enables the whole array of features: `AssetRegistry`, `AssetVault`, `AMM`, `ProtocolGovernor`, ERC-20 token via locking collaterals in ERC-4626 vault, trading of the asset in customized constant product automated market maker and finally, governing of all privilege actions via token-weighted Governor with `TimelockController` two-day delay mechanism.

System involves nine solidity contracts, two test-only mocks, three deployment scripts, and invariant/fuzz harness of 192 lines. The protocol is currently deployed on Base Sepolia with all four indexed contracts live on-chain (`AssetRegistry`, `AssetVault`, `AMM`, `ProtocolGovernor`) and an active subgraph indexing five entity types. The repository reports **82 passing Foundry tests**, **96.64 % line coverage** and a zero-finding Slither snapshot.

Architectural philosophy. Every state-changing privileged operation must be reachable only through the `TimelockController`; the deployer EOA is treated as a strictly ephemeral bootstrapper and is stripped of all admin roles inside the same broadcast block. The hand-off is verified by a dedicated post-deploy assertion script.

1.1 Component inventory at a glance

Layer	Contract	Purpose	Standards / Patterns
Governance	GovernanceToken	Token-weighted voting	ERC-20 · Permit · Votes · Ownable · cap 100 M
	ProtocolGovernor	Proposal / voting / queue / execute	OZ Governor + Settings + CountingSimple + VotesQuorumFraction + TimelockControl
Upgradeable	AssetRegistry / AssetRegistryV2	Issuer authorisation, asset metadata	UUPSUpgradeable · AccessControl · ERC1967Proxy
Asset	AssetToken	ERC-20 representation of the RWA	ERC-20 · Burnable · Permit · Ownable · AccessControl · cap 1 B
	AssetReceipt	ERC-1155 receipt layer	ERC1155Supply · AccessControl · per-id supply cap
	AssetVault	Reserve-backed 1:1 mint / redeem	ERC-4626 · Ownable · ReentrancyGuard
	VaultFactory	Deterministic / non-deterministic vault deployment	CREATE + CREATE2
Market	AMM	Constant-product swap pair	$x \cdot y = k$ · 30 bps fee · LP token · ReentrancyGuard · Yul sqrt
	PriceOracle	Chainlink-compatible price feed wrapper	AggregatorV3 · stale-window · round validation

1.2 Cross-stack composition

The smart-contract stack is integrated with off-chain components through two parallel data paths. Direct JSON-RPC reads/writes are issued by the React frontend via **wagmi** / **viem** against an RPC provider; historical and analytical queries are served by a deployed subgraph indexing every **Proposal**, **Vote**, **Asset**, **VaultPosition**, and **Swap** emitted by the four indexed contracts. Verification artefacts are surfaced through Etherscan API, and end-to-end behaviour is provable by the **VerifyPostDeploy.s.sol** assertion script.

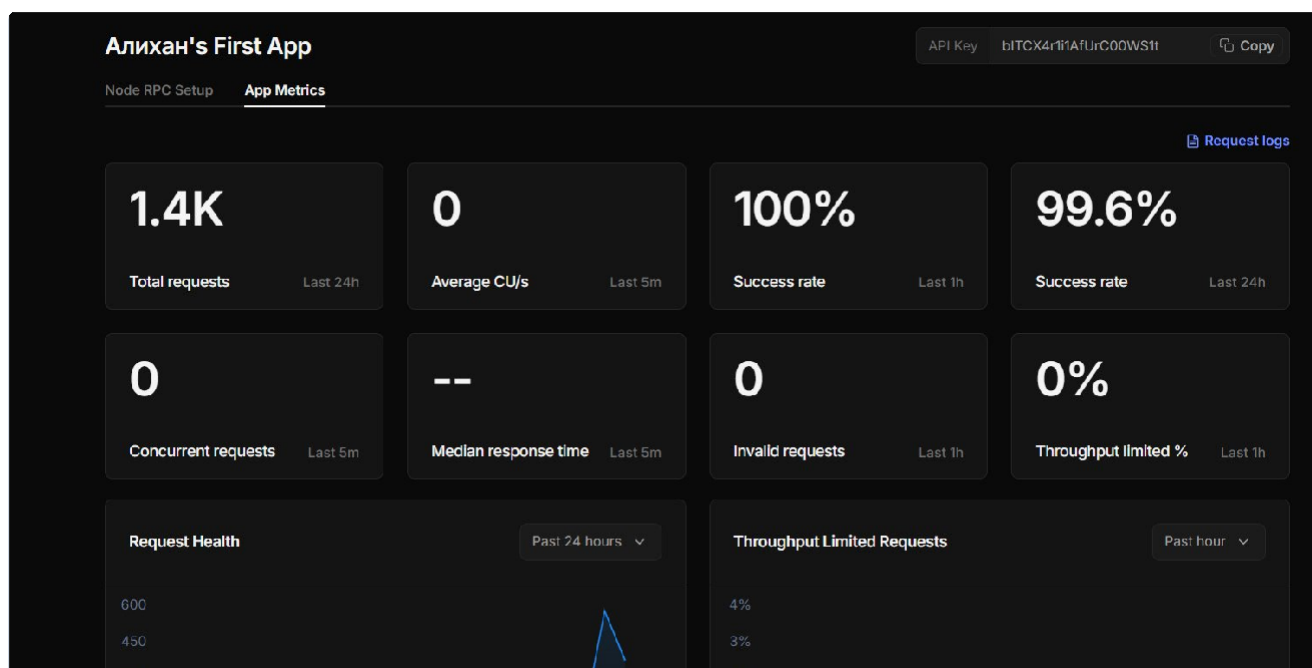


Figure 1.1 — Live RPC traffic against the deployed protocol over a 24-hour window: 1.4 K requests, 100 % 1-hour success rate, 99.6 % 24-hour success rate, 0 % throughput-limited responses (operator dashboard, QuickNode “App Metrics”).

It is a testnet deployment, since the economic values used are optimal for the block time of the Base Sepolia network (delay for vote creation 43 200 blocks = ~1 day, duration of votes 302 400 blocks = ~7 days, proposal fee threshold 10 000 GOV (=1% of the total supply), quorum 4%, minimum delay for timelock transactions 2 days). There is bootstrap liquidity supplied by the deployment script (ratio 10 000/10 000, 5 000 unit vault position).

2. System Overview

This design creates five orthogonal layers in the tokenization space for the RWA tokens, implemented using a set of smart contracts with specific functions. This design achieves the goal deliberately in order to keep onboarding privileged from accounting, secondary market trading different from issuing, and upgrading constrained to one proxy layer only.

Surface	Primary actor	Responsibility	Backing contracts
1. Asset onboarding	Issuer (governance-approved)	Register an RWA, attach metadata URI, set max supply	<code>AssetRegistry</code> (proxy)
2. Issuance / redemption	Investor	Lock backing collateral and mint a 1:1 ERC-20 representation	<code>AssetVault</code> , <code>AssetToken</code> , <code>AssetReceipt</code>
3. Secondary trading	Trader / LP	Exchange asset tokens against a quote token with slippage protection	<code>AMM</code>
4. Governance	Token holder / Voter	Authorise issuers, pause assets, upgrade registry, rotate parameters	<code>GovernanceToken</code> , <code>ProtocolGovernor</code> , <code>TimelockController</code>
5. Off-chain UX	Any user	Read aggregated state and submit signed transactions	<code>frontend/</code> , <code>subgraph/</code>

2.1 End-to-end value flow

A complete lifecycle from issuance to redemption involves all five surfaces. Governance authorises an issuer, the issuer registers the asset, an investor deposits collateral into the vault to mint asset-tokens, those tokens become tradable on the AMM, and at any point the investor (or a holder who acquired the token on the secondary market) can return to the vault to burn-and-redeem against the underlying collateral. The vault guarantees that `assetToken.totalSupply() == vault.totalDeposited()` at all times — an invariant enforced both in code (via the `_recordDeposit` / `_recordWithdrawal` hooks) and in the test harness (via `invariant_VaultSupplyConservation`).

2.2 Trust boundaries

- Smart-contract boundary.** All state-changing logic runs on Base Sepolia. The deployer EOA crosses this boundary only during the `Deploy.s.sol` broadcast, after which it holds zero admin roles (asserted by `VerifyPostDeploy.s.sol`).
- Governance boundary.** Any mutation of registry permissions, asset metadata, implementation address, or token ownership must transit the `TimelockController` after a 2-day delay; arbitrary callers may execute already-queued operations (executor role is open).

3. **Oracle boundary.** The `PriceOracle` wraps an external AggregatorV3 feed. The contract trusts feed-reported values within a configured `staleAfter` window and explicitly rejects negative prices, unset rounds, and incomplete rounds.
4. **Off-chain boundary.** The frontend reads chain state via JSON-RPC and subgraph state via GraphQL. It does not custody keys; signatures originate in the user's wallet.

3. System Context (C4 Level 1)

The figure 3.1 below locates the protocol within the context of operation. Four actor groups interact with the protocol, three different software components make up the product, and four infrastructure vendors provide the network/data services.

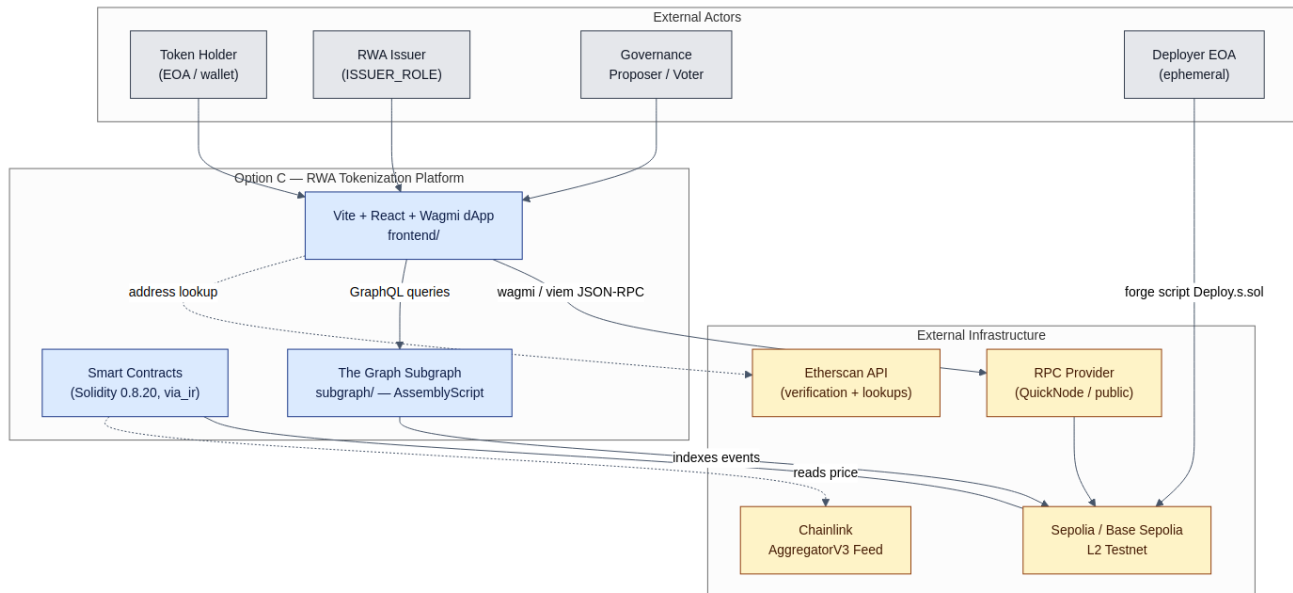


Figure 3.1 — C4 Level 1 system context. Solid arrows are transactional dependencies; dotted arrows are read-only lookups.

3.1 External actors

Actor	Role	Interfaces with
Token holder (investor)	EOA / wallet	Frontend → Vault, AMM, GovernanceToken
RWA issuer	Holds ISSUER_ROLE on Registry (granted by governance)	Frontend → AssetRegistry.registerAsset
Governance proposer / voter	Holds $\geq 10\,000$ GOV after self-delegation	Frontend → Governor (propose / castVote / queue / execute)
Deployer EOA	Ephemeral bootstrapper	forge script Deploy.s.sol

3.2 External infrastructure

Provider	Function	Failure mode handled
Chainlink AggregatorV3 feed	Spot price for oracle wrapper	Stale / negative / incomplete-round → revert with custom error
RPC provider (QuickNode / public)	JSON-RPC reads and writes	Frontend falls back to chain's default RPC if <code>VITE_PUBLIC_RPC_URL</code> unset
Base Sepolia testnet	Execution layer	Chain ID mismatch surfaced to UI via <code>useChainId()</code>
Etherscan API	Contract verification, source code lookup	Not on the critical path

ETH Price: \$2,169.17 (-2.66%) Gas: 0.2 Gwei

Search by Address / Txn Hash / Block / Token / Domain

The last 25 calls made with the API Key: AE2EDGPH2XKED51V4IMGE8K1EQBF21ZA8R

Timestamp (UTC)	Referrer IP	Chain ID	Module	Action	Data
05/17/2026 00:12:14	147.30.172.125	11155111	contract	getsourcecode	0x00
05/17/2026 00:01:19	147.30.172.125	11155111	contract	getsourcecode	0x9cb7741e7f8661173e3ef215de82a563c8effb0d
05/17/2026 00:01:19	147.30.172.125	11155111	contract	getsourcecode	0x9f114a7f9af9aab86de10c1a43ac3cac482388b
05/16/2026 23:56:45	147.30.172.125	11155111	contract	getsourcecode	0x0104798a09428aa7efcc0bfbcd17d8301deca023
05/16/2026 23:56:45	147.30.172.125	11155111	contract	getsourcecode	0xe1071ad02d88fbfb1f6998a4aff28b641cbb7e24
05/16/2026 23:56:45	147.30.172.125	11155111	contract	getsourcecode	0xc3b2592aa4901b9fc621668d493f736aee2ed10d
05/16/2026 23:51:00	147.30.172.125	11155111	contract	getsourcecode	0xd135a3e3094064d196defbd17291ce388584d109
05/16/2026 23:51:00	147.30.172.125	11155111	contract	getsourcecode	0x9a5f0df96c9333bc81f56c4cab7956d9d47a298a
05/16/2026 23:51:00	147.30.172.125	11155111	contract	getsourcecode	0xbc8bb735cd83efa5439481b639014dac4cee1092

Figure 3.2 — Etherscan API Dashboard showing live `getsourcecode` requests against the deployed contracts (chain ID 11155111 = Sepolia). These calls support the frontend's verification banner and the developer workflow.

4. Container & Component Architecture

In figure 4.1, the smart-contract box is deconstructed into its individual parts and how they interact on the chain. The colour-coded groups represent the five different system surfaces identified in §2, and the arrows represent the interface between them.

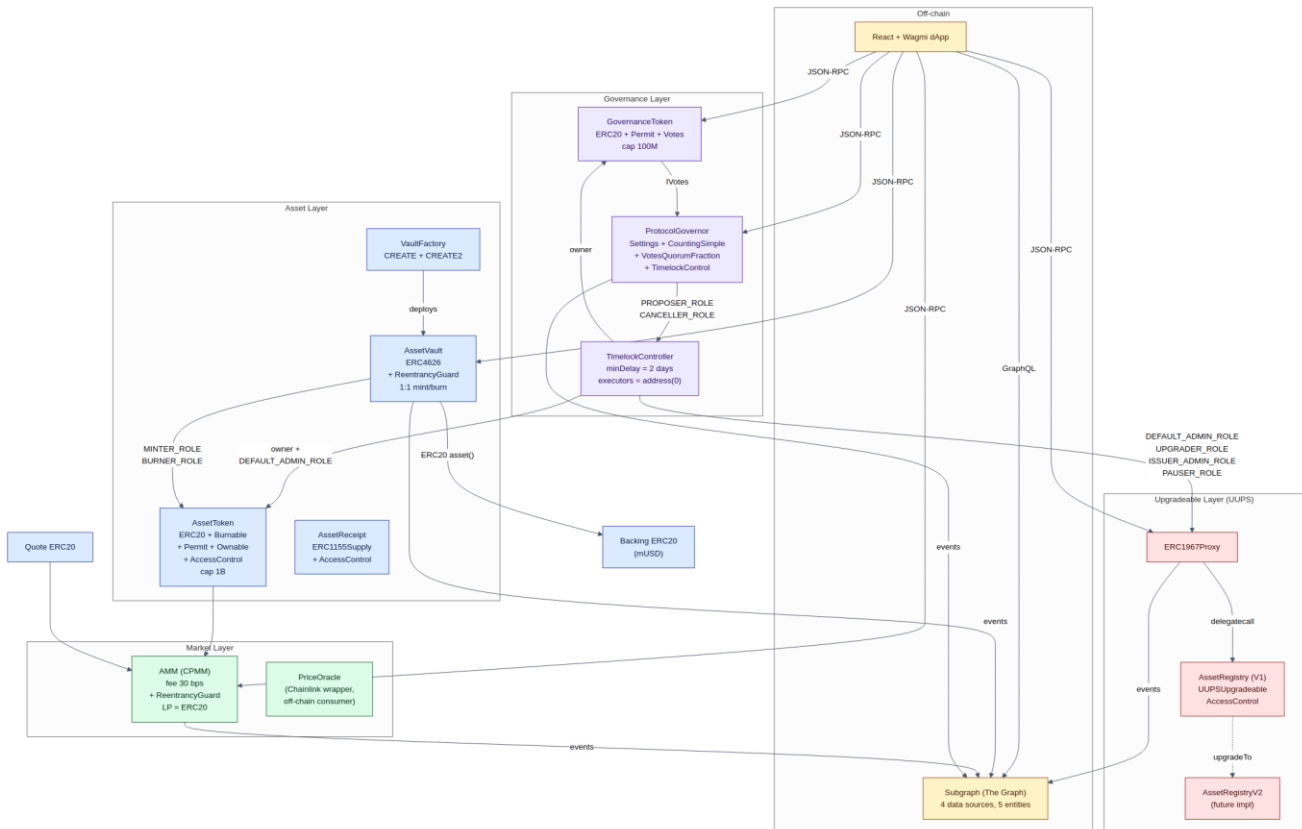


Figure 4.1 — Container / component architecture with inheritance, role grants, and event flows.

4.1 Inheritance hierarchies

Contract	Direct parents	Notes
<code>AssetRegistry</code>	<code>AccessControl</code> , <code>UUPSUpgradeable</code>	Custom <code>bool _initialized</code> guard (no OZ <code>Initializable</code>)
<code>AssetRegistryV2</code>	<code>AssetRegistry</code>	Append-only – adds <code>version()</code> and <code>pingV2()</code> ; no storage reordering
<code>AssetToken</code>	<code>ERC20</code> , <code>ERC20Burnable</code> , <code>ERC20Permit</code> , <code>Ownable</code> , <code>AccessControl</code>	Owner and <code>MINTER_ROLE</code> / <code>BURNER_ROLE</code> are parallel privilege paths
<code>AssetReceipt</code>	<code>ERC1155Supply</code> , <code>AccessControl</code>	Per-token-id <code>maxSupply</code> + <code>_tokenURIs</code> override
<code>AssetVault</code>	<code>ERC4626</code> , <code>Ownable</code> , <code>ReentrancyGuard</code>	All four ERC-4626 entrypoints overridden to mint/burn the asset token
<code>AMM</code>	<code>ERC20</code> , <code>ReentrancyGuard</code> , <code>Ownable</code>	LP token is the AMM contract itself
<code>GovernanceToken</code>	<code>ERC20</code> , <code>ERC20Permit</code> , <code>ERC20Votes</code> , <code>Ownable</code>	Snapshot semantics inherited from <code>ERC20Votes</code>
<code>ProtocolGovernor</code>	<code>Governor</code> , <code>GovernorSettings</code> , <code>GovernorCountingSimple</code> , <code>GovernorVotes</code> , <code>GovernorVotesQuorumFraction</code> , <code>GovernorTimelockControl</code>	All required <code>override</code> s implemented; supports <code>IERC165</code>

4.2 Proxy & factory relationships

`AssetRegistry` is deployed behind an `ERC1967Proxy`; the proxy is the publicly addressable entity (`0xbc8Bb735...` on Base Sepolia), while the implementation contract is referenced only via the EIP-1967 implementation-address slot. Upgrades are authorised by `_authorizeUpgrade(newImpl)`, gated by `UPGRADER_ROLE` and explicitly rejecting the zero address.

`VaultFactory` deploys additional `AssetVault` instances on demand, exposing both a non-deterministic `createVault` (CREATE opcode) and a deterministic `createVaultDeterministic(salt)` (CREATE2). The address-prediction helper `predictVaultAddress(...)` reconstructs the CREATE2 digest from `type(AssetVault).creationCode` and constructor args, enabling off-chain verification before deployment.

4.3 Event surface (consumed by the subgraph)

Contract	Indexed event	Subgraph entity touched
AssetRegistry	AssetRegistered(bytes32, address, address, address, address, uint256, string)	Asset (created)
AssetRegistry	AssetActiveSet(bytes32, bool)	Asset.active
AssetVault	DepositRecorded(address, uint256)	VaultPosition.deposited += amount
AssetVault	WithdrawalRecorded(address, uint256)	VaultPosition.deposited -= amount
AssetVault	TokensMinted(address, uint256)	VaultPosition.assetTokenMinted += amount
AMM	Swap(address, uint256, uint256, bool)	Swap (created)
ProtocolGovernor	ProposalCreated(...), VoteCast(...), ProposalQueued, ProposalExecuted, ProposalCanceled	Proposal, Vote

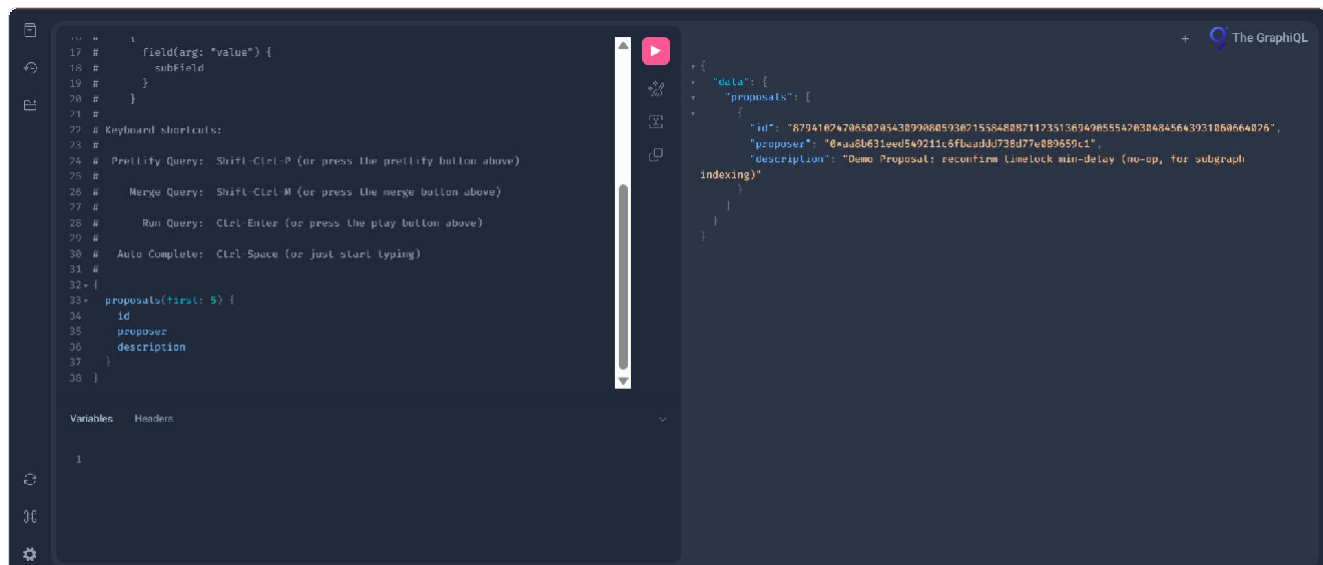


Figure 4.2 — GraphQL session against the deployed subgraph showing a successful `proposals(first: 5)` query. The single result is the live no-op governance proposal created by `script/CreateProposal.s.sol` (“Demo Proposal: reconfirm timelock min-delay (no-op, for subgraph indexing)”), confirming end-to-end indexer functionality.

5. Access Control Model

The protocol uses OpenZeppelin's `AccessControl` with eleven named roles across three contracts, plus the `Ownable` owner slot on `AssetToken`, `AssetVault`, `AMM`, and `GovernanceToken`. After `Deploy.s.sol` completes, every privileged role is held by the `TimelockController` or by a contract that itself answers to the timelock.

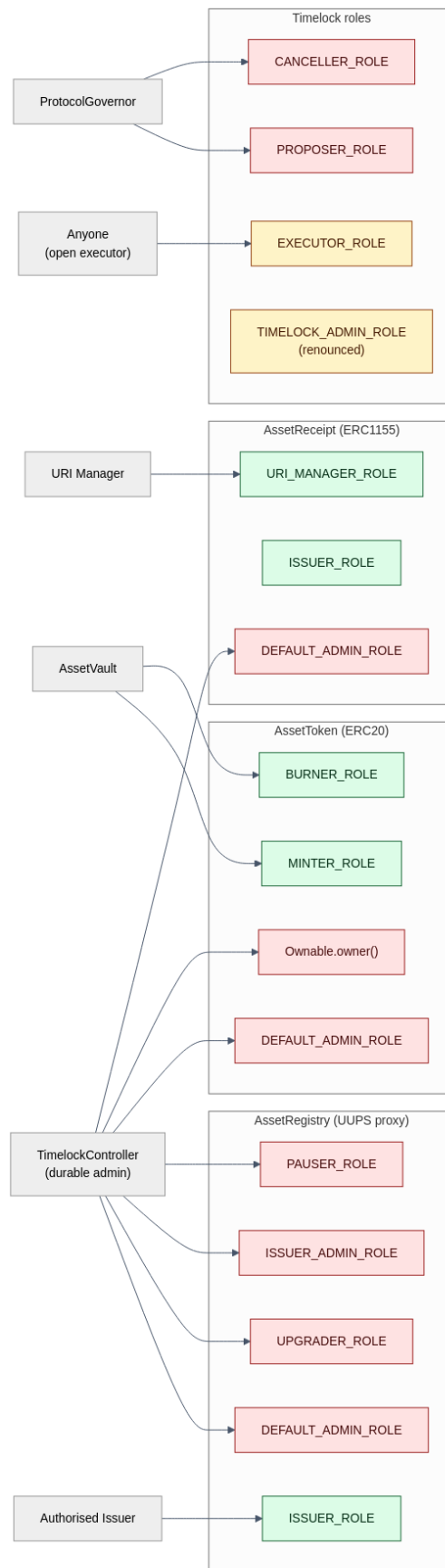


Figure 5.1 — Final role assignments after deployment hand-off. Red boxes mark privileged roles; green marks roles delegated to a specific operational contract; yellow indicates either an open executor or a deliberately renounced role.

5.1 Role definitions

Contract	Role / slot	Holder after deployment	Powers granted
AssetRegistry	DEFAULT_ADMIN_ROLE	TimelockController	Grant / revoke all other registry roles
	UPGRADER_ROLE	TimelockController	Authorise UUPS upgrade target
	ISSUER_ADMIN_ROLE	TimelockController	Authorise / revoke individual ISSUER_ROLE grants
	ISSUER_ROLE	Approved issuers (per asset)	Call registerAsset(...)
	PAUSER_ROLE	TimelockController	Toggle active flag per asset
AssetToken	DEFAULT_ADMIN_ROLE	TimelockController	Grant / revoke MINTER_ROLE / BURNER_ROLE
	MINTER_ROLE	AssetVault	Call mint(to, amount)
	BURNER_ROLE	AssetVault	Call burnBackingFrom(account, amount)
	Owable.owner()	TimelockController	Parallel privilege path for mint/ burn-backing checks
AssetReceipt	DEFAULT_ADMIN_ROLE	Admin (constructor arg)	Grant / revoke other receipt roles
	ISSUER_ROLE	Admin (constructor arg)	Mint ERC-1155 receipt units
	URI_MANAGER_ROLE	Admin (constructor arg)	Configure per-id max supply + URI
TimelockController	TIMELOCK_ADMIN_ROLE	renounced by deployer	(none – would have managed delay & roles)
	PROPOSER_ROLE	ProtocolGovernor	Schedule queued operations
	CANCELLER_ROLE	ProtocolGovernor	Cancel a queued operation
	EXECUTOR_ROLE	address(0) — open	Anyone may execute a queued op after delay
GovernanceToken	Owable.owner()	TimelockController	Call mint(to, amount) (cap-bounded)

5.2 Trust assumptions

1. The `TimelockController` is honest and not the target of a >50 % voting-power capture, because all `DEFAULT_ADMIN_ROLE` grants flow through it.
2. The `AssetVault` deployed by the deployment script is the only address that ever needs `MINTER_ROLE` / `BURNER_ROLE` on `AssetToken`. Any future vault must be granted these roles by a governance proposal.
3. Once `VerifyPostDeploy.s.sol` succeeds, the deployer key is no longer privileged anywhere in the system and may be discarded.
4. The open `EXECUTOR_ROLE` is intentional: it removes an operational bottleneck and is safe because the executor can only execute operations that the Governor already queued.

6. Governance Lifecycle

The full proposal lifecycle from delegation to execution is implemented through OpenZeppelin's Governor extensions and the `TimelockController`. Figure 6.1 shows the canonical happy-path with the on-chain state of the proposal annotated for each phase. All five phases — delegation, creation, voting, queuing, execution — are exercised end-to-end by `testFullGovernorTimelockExecution` in `test/Governance.t.sol`.

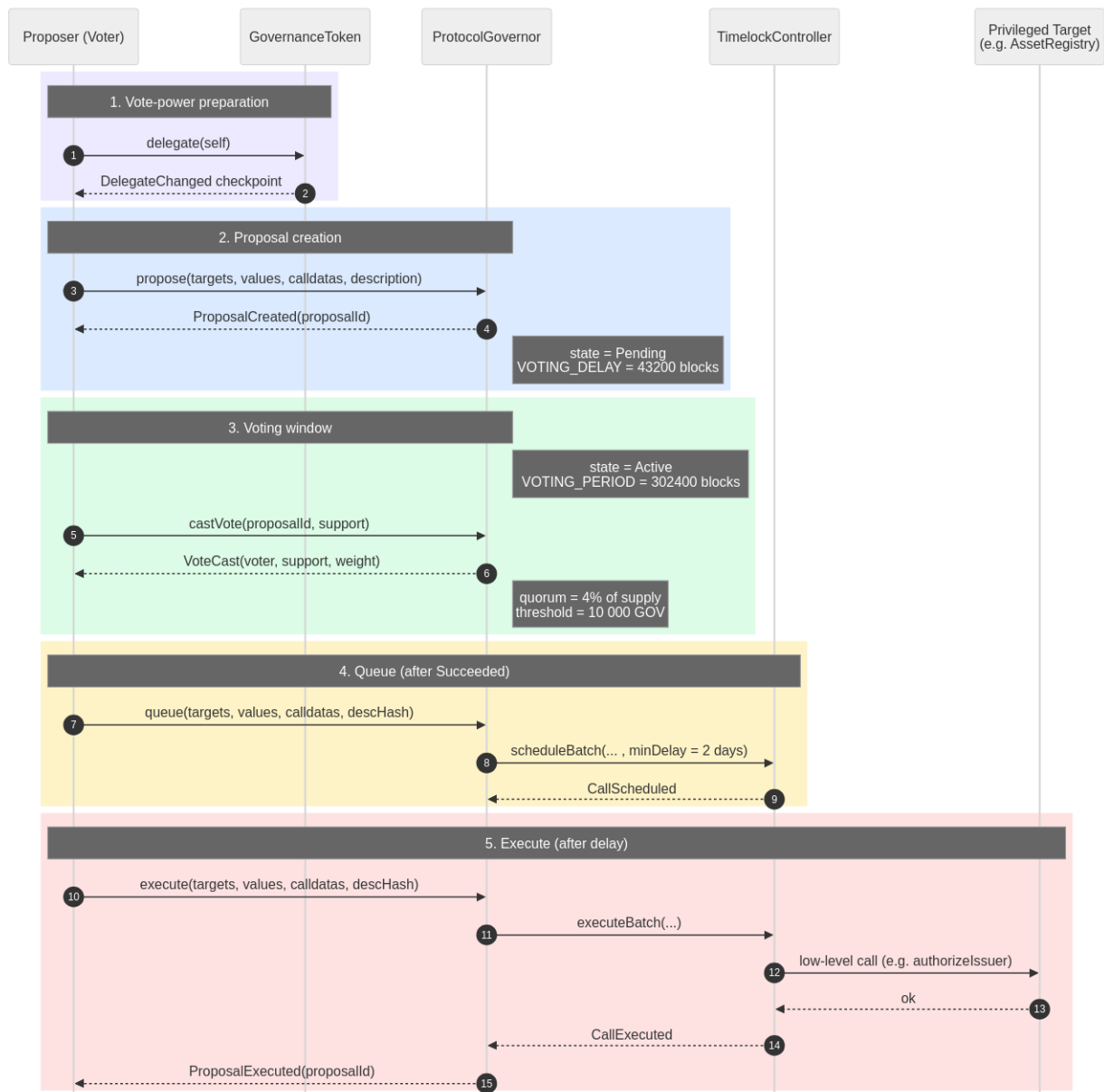


Figure 6.1 — Full proposal lifecycle. Colour bands map onto the five phases; the legend on the right of each band lists the relevant Governor parameter.

6.1 Governance parameters (constants in `ProtocolGovernor`)

Constant	Value	Translation (Base Sepolia, 2 s blocks)	Justification
<code>VOTING_DELAY_BLOCKS</code>	43 200	≈ 24 h	Gives token holders one day to react before voting opens
<code>VOTING_PERIOD_BLOCKS</code>	302 400	≈ 7 days	One full week to gather participation
<code>PROPOSAL_THRESHOLD</code>	$10\,000 \times 10^{18}$	1 % of initial 1 M circulating supply	Mitigates proposal spam from low-stake actors
<code>QUORUM_PERCENT</code>	4	4 % of total supply at proposal snapshot	Standard Compound/OZ default; balances participation vs liveness
<code>TimelockController.minDelay</code>	2 days	2 days	Window for stakeholders to exit if a malicious proposal queues

6.2 Demo proposal — observed in production

The script `script/CreateProposal.s.sol` creates a deliberately benign proposal: `timelock.updateDelay(currentDelay)` . The call is a no-op (the delay is set to its current value) and exists so the subgraph has at least one indexed `ProposalCreated` event to query against. Figure 4.2 (above) shows the resulting proposal returned by the live subgraph, proving the entire event-flow pipeline works end-to-end on the deployed system.

Two-phase script design. The script gracefully handles the `ERC20Votes` checkpoint delay: on the first run it self-delegates and exits; on the second run it observes the new voting power, validates against `proposalThreshold()` , and submits the proposal. This pattern is documented in the script header and is a small but practical UX detail.

7. AMM Swap Flow

The custom AMM implements the canonical Uniswap V2 constant-product invariant $x \cdot y = k$, with a fixed 30-basis-point fee and slippage protection on every swap entry-point. The contract is its own LP token (`ERC20("LP Token", "LP")`), which simplifies state but means the AMM cannot be used as collateral by another protocol expecting an external LP token contract.

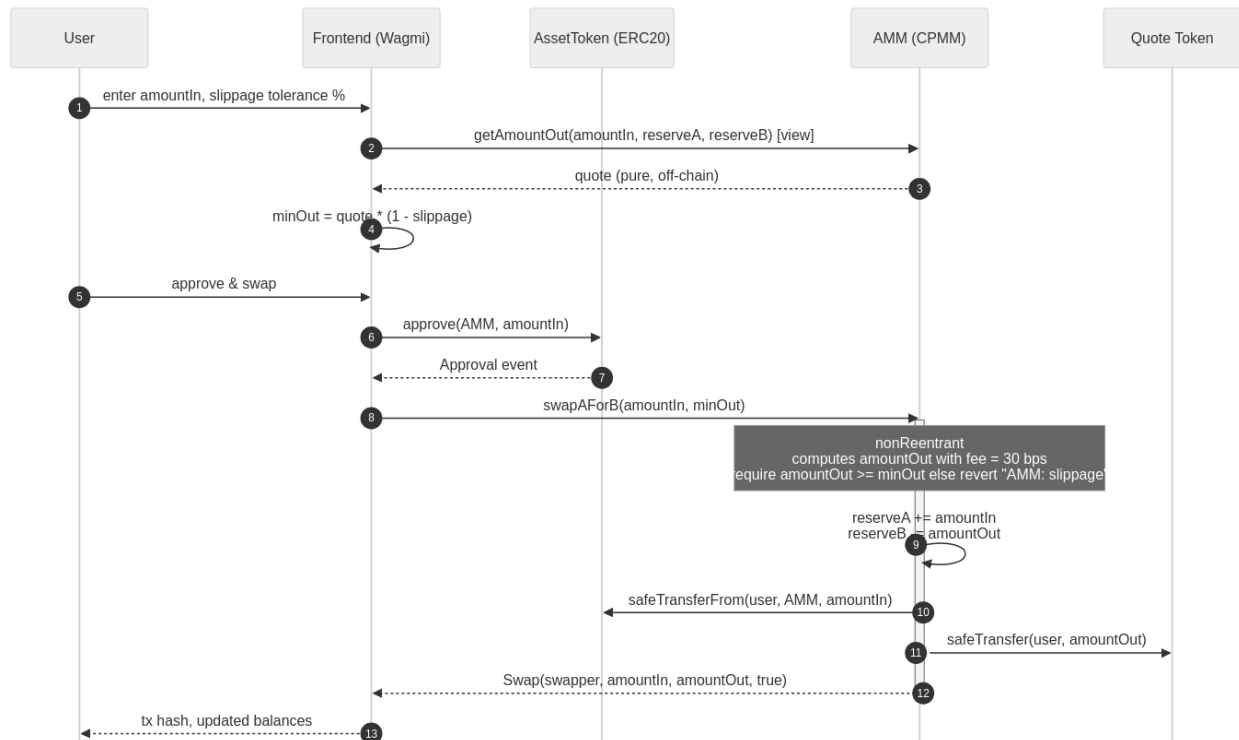


Figure 7.1 — AMM `swapAForB` happy-path. The frontend always computes an off-chain quote and applies a slippage tolerance before submitting.

7.1 Pricing math

The `getAmountOut` implementation is mathematically equivalent to Uniswap V2's:

```

amountInWithFee = amountIn * (10_000 - 30)
amountOut       = (amountInWithFee * reserveOut)
                  / (reserveIn * 10_000 + amountInWithFee)
  
```

A 0.30 % swap fee is implicitly retained in the pool as the residual difference between `amountIn` and `amountInWithFee / (BPS_DENOMINATOR)`, accruing to existing LP holders pro-rata via `reserveA / reserveB` growth. The function is `pure` so it is safe to call from the frontend without incurring a state read on the live reserves.

7.2 Reentrancy & CEI

- All four state-mutating entry-points (`swapAForB`, `swapBForA`, `addLiquidity`, `removeLiquidity`) carry the `nonReentrant` modifier.

- Reserve state is updated *before* the external `safeTransferFrom` and `safeTransfer` calls in the swap functions, enforcing checks-effects-interactions even though the modifier already guards the contract.
- The invariant `tokenX.balanceOf(amm) == reserveX` is asserted continuously by `invariant_ReservesMatchTokenBalances` in the fuzz harness, with 32 runs × 64 calls per invariant.

7.3 Slippage protection

Each swap function exists in two overloads. The single-argument overload defaults `minAmountBOut = 0` (no protection); the two-argument overload requires the caller to supply a minimum. The frontend always uses the two-argument form, computing `minOut = quote × (1 - slippageTolerance)` client-side. On-chain callers who use the single-argument form expose themselves to sandwich attacks; this is documented in §9 of the audit report.

7.4 Initial liquidity bootstrap

For the first liquidity provision (`reserveA == 0 && reserveB == 0`) the LP supply minted is $\sqrt{(\text{amountA} \times \text{amountB})}$, computed by an inline-assembly Babylonian square-root (`sqrAssembly`). The function is benchmarked against a pure-Solidity equivalent (`sqrSolidity`) by `testGasBenchmarkSqrAssemblyVsSolidity` ; on the recorded snapshot the assembly version uses **4 109 gas vs 17 335**, a **76 % reduction**.

8. Vault Deposit & Redeem Flow

AssetVault is the protocol's reserve engine. Every deposit of the backing ERC-20 (**mUSD** in the demo deployment) mints an equal quantity of two distinct tokens: ERC-4626 **shares** (**rvASSET** , used for ERC-4626 accounting) and the **AssetToken** (the protocol's tradable ERC-20 representation). The vault therefore acts as both an ERC-4626 share issuer and a sovereign mint for the protocol's external token.

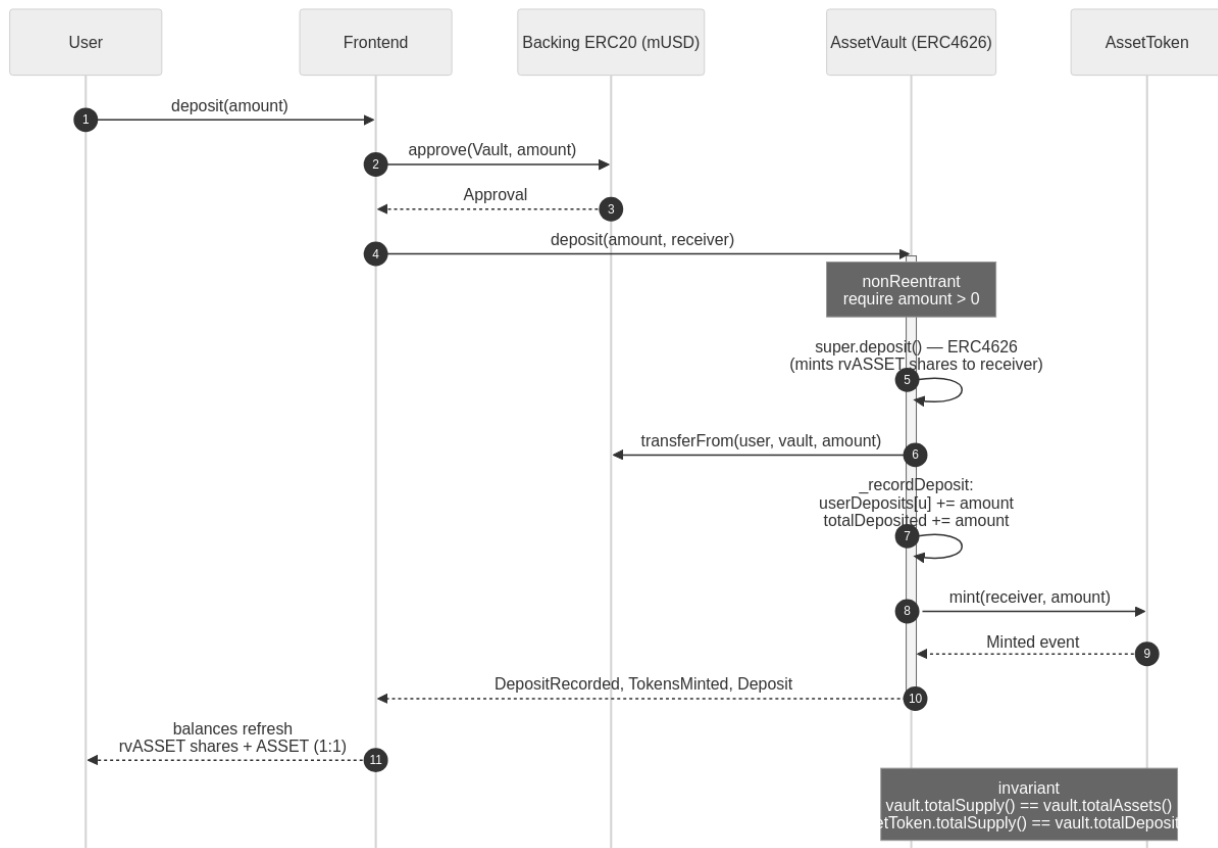


Figure 8.1 — Deposit flow. **nonReentrant** guards the entire path; per-user and global accounting is recorded before the asset-token mint.

8.1 Mint / burn parity

For each unit of backing collateral deposited, exactly one unit of **AssetToken** is minted to the receiver; redemption burns the same amount. This guarantees a structural 1:1 backing ratio independent of the share-price drift that a general-purpose ERC-4626 vault would experience. The invariants are:

Invariant	Enforced by	Test
<code>vault.totalSupply() == vault.totalAssets()</code>	<code>_depositReserve / _withdrawReserve</code>	<code>invariant_VaultSupplyConservation</code>
<code>assetToken.totalSupply() == vault.totalDeposited()</code>	same	<code>invariant_VaultSupplyConservation</code>
<code>reserve.balanceOf(vault) == vault.totalAssets()</code>	ERC-4626 + ReentrancyGuard	<code>invariant_VaultTreasuryAccountingMatchesReserveBalance</code>
<code>getReserveRatio() >= 100</code> when deposited > 0	1:1 parity above	<code>invariant_VaultReserveRatioHealthyWhenDeposited</code>

8.2 Withdrawal pre-conditions

A user can only withdraw collateral they actually deposited: `require(userDeposits[owner_] >= assets, "AssetVault: insufficient deposit")`. The asset token is burned *before* the ERC-4626 `super.withdraw` moves collateral out, again enforcing CEI on top of the reentrancy guard. The withdrawal path calls `assetToken.burnBackingFrom(owner_, assets)`, which itself requires the vault to hold `BURNER_ROLE` or be the owner of the asset token — both of which are established in the deployment script.

8.3 Edge cases handled

- Zero-amount deposit or redeem reverts with `"AssetVault: zero assets"`.
- Withdraw on behalf of a third party respects the ERC-4626 allowance check inherited from `super.withdraw`.
- The `mint(shares, receiver)` override pre-computes `previewMint(shares)` and rejects zero-asset paths.

9. Data Model & Storage Layout

This section enumerates the storage variables of each first-party contract, identifies slot collisions or fragility concerns, and discusses upgrade safety for the only upgradeable contract (`AssetRegistry`).

9.1 `AssetRegistry` (upgradeable)

Slot	Variable	Type	Purpose
0... N	Inherited <code>AccessControl._roles</code>	<code>mapping(bytes32 ⇒ RoleData)</code>	OZ role storage
N+1	<code>_initialized</code>	<code>bool</code>	Custom initializer guard
N+2	<code>assetCount</code>	<code>uint256</code>	Number of registered assets
N+3	<code>_assets</code>	<code>mapping(bytes32 ⇒ AssetRecord)</code>	Asset metadata, keyed by <code>assetId</code>

The struct `AssetRecord` contains `{ address issuer; address reserveAsset; address assetToken; address vault; uint256 maxSupply; string metadataURI; bool active; }`. Strings are stored in a separate slot (per Solidity packing rules) so the struct does not overflow the dynamic-key mapping pattern.

Upgrade-safety note. The contract does **not** reserve a `__gap` array. Future revisions must append new variables only, never re-order, and must not introduce additional inherited base contracts without verifying the layout against `AssetRegistry` 's slot sequence. The repository's `AssetRegistryV2` respects this constraint (it appends a single event and two function definitions – no storage).

9.2 `AssetToken`

Variable	Type	Notes
<code>MINTER_ROLE</code> , <code>BURNER_ROLE</code>	<code>bytes32</code> <code>constant</code>	Compile-time – no slot
<code>MAX_SUPPLY</code>	<code>uint256</code> <code>constant</code>	Hard cap at $1 \text{ B} \times 10^{18}$
Inherited	—	<code>ERC20</code> balances + allowances, <code>ERC20Permit</code> nonces + domain, <code>AccessControl._roles</code> , <code>Ownable._owner</code>

9.3 `AssetVault`

Variable	Type	Purpose
<code>assetToken</code>	<code>AssetToken immutable</code>	No slot – bytecode constant
<code>totalDeposited</code>	<code>uint256</code>	Cumulative active collateral
<code>RESERVE_RATIO</code>	<code>uint256 constant</code>	Hard-coded 100 (i.e. 100 % backing required)
<code>userDeposits</code>	<code>mapping(address ⇒ uint256)</code>	Per-user withdrawal budget

9.4 `AMM`

Variable	Type	Purpose
<code>tokenA</code> , <code>tokenB</code>	<code>IERC20</code> <code>immutable</code>	Trading pair (bytecode constants)
<code>reserveA</code> , <code>reserveB</code>	<code>uint256</code>	Current pool reserves
<code>FEE_BPS</code> , <code>FEE_PERCENT</code> , <code>BPS_DENOMINATOR</code>	<code>uint256</code> <code>constant</code>	Fee math; <code>FEE_PERCENT == FEE_BPS</code> is a redundant alias

9.5 `GovernanceToken` and `ProtocolGovernor`

Variable	Type	Notes
<code>MAX_SUPPLY</code>	<code>uint256</code> <code>constant</code>	$100 \text{ M} \times 10^{18}$
Governor constants	<code>uint48</code> , <code>uint32</code> , <code>uint256</code> constants	See §6.1
Inherited	—	<code>ERC20Votes</code> checkpoints, <code>Governor</code> proposal map, <code>GovernorTimelockControl</code> queued ops

9.6 `PriceOracle`

Both state variables are `immutable` and live in bytecode rather than storage: `feed` (the AggregatorV3 address) and `staleAfter` (the freshness window in seconds). The contract has therefore no mutable storage surface at all.

10. Upgradeability Architecture

Only `AssetRegistry` is upgradeable. The protocol intentionally keeps the financial layer (`AssetVault` , `AssetToken` , `AMM`) non-upgradeable to provide strong immutability guarantees for users; any change to those contracts requires a fresh deployment and a governance proposal to re-wire roles.

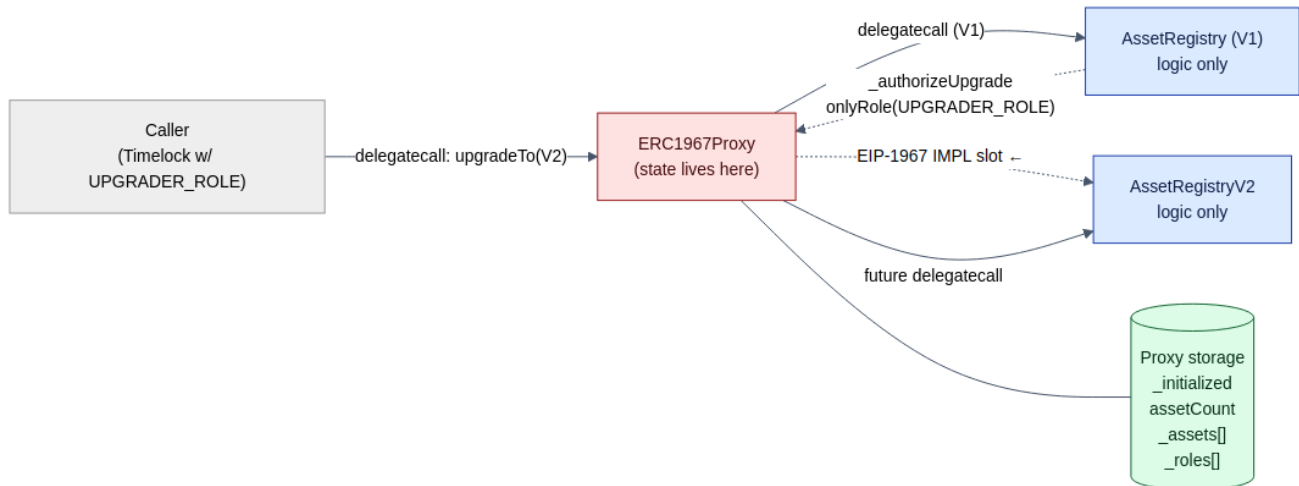


Figure 10.1 — UUPS pattern. The proxy is the source of truth for state; the implementation contributes logic only. Authorisation is enforced inside the implementation, not the proxy.

10.1 Pattern choice — UUPS vs Transparent

The project uses **UUPS** (OpenZeppelin's `UUPSUpgradeable`) rather than the Transparent proxy pattern. This is the right choice for this protocol because: (a) the proxy itself contains no admin logic and is therefore minimal; (b) the upgrade authorisation lives in the implementation, gated by `UPGRADER_ROLE`; (c) gas costs on every call are lower (no admin-vs-user branching in the proxy).

10.2 Initialization

Aspect	Implementation choice	Trade-off
Initializer guard	Custom <code>bool _initialized</code>	Simpler, but does not protect against re-initialization via inheritance and does not match OZ's <code>Initializable</code> conventions
Implementation lock	Not invoked in <code>Deploy.s.sol</code>	The raw implementation contract can still have <code>initialize()</code> called on its own storage — minor risk (state lives in the proxy), discussed in audit finding L-01
Upgrade authorisation	<code>_authorizeUpgrade(newImpl)</code> <code>onlyRole(UPGRADER_ROLE)</code>	Standard; the role is held by the Timelock, so an upgrade requires a 2-day-delayed governance proposal
Storage gap	Absent	Future revisions must append, not insert; documented in §9.1

10.3 Upgrade path observed in tests

`testRegistryUpgradeToV2` deploys `AssetRegistryV2`, calls `upgradeTo(v2)` on the proxy, and verifies that the new function `version()` returns `"2.0.0"` while the original storage (`assetCount`) remains intact. This is a representative regression test for the storage-safety contract.

11. Oracle Architecture

The PriceOracle component of the protocol represents the Chainlink-based oracle module. The module is responsible for the price validation of the AggregatorV3 prior to its exposure to the frontend and the test suite. It discards any invalid prices, negative or undefined answers, unset or unfinished rounds. Currently, the oracle is used defensively to validate prices and is covered by tests for the mock-aggregator.

11.1 Validation pipeline

Check	Condition	Revert
Sign	<code>price > 0</code>	<code>InvalidPrice()</code>
Round started	<code>startedAt != 0</code>	<code>InvalidPrice()</code>
Round complete	<code>answeredInRound >= roundId</code>	<code>IncompleteRound(roundId, answeredInRound)</code>
Freshness	<code>block.timestamp - updatedAt <= staleAfter</code>	<code>StalePrice(updatedAt, currentTime)</code>

11.2 Trust model

- The protocol fully trusts the Chainlink feed's reported value within the freshness window.
- There is **no multi-oracle aggregation**, no median, and no fallback feed. A compromised or deviating feed will pass all four checks if it still reports a positive, complete, freshly-updated value.
- The freshness window `staleAfter` is set at deployment and is `immutable`; rotating it requires deploying a new oracle.

Staleness uses `block.timestamp` . Slither typically flags this as a timestamp-dependency warning. In this context the warning is benign because the comparison tolerates the small (≤ 15 s) drift a validator can induce, and a stale window of 1 h (the test default) or longer absorbs that drift comfortably. The use is explicitly annotated with `slither-disable-next-line timestamp` .

12. The Graph & Indexing Architecture

The subgraph is the protocol's source of truth for historical state and aggregated reads. It indexes four on-chain data sources and exposes five entity types via GraphQL.

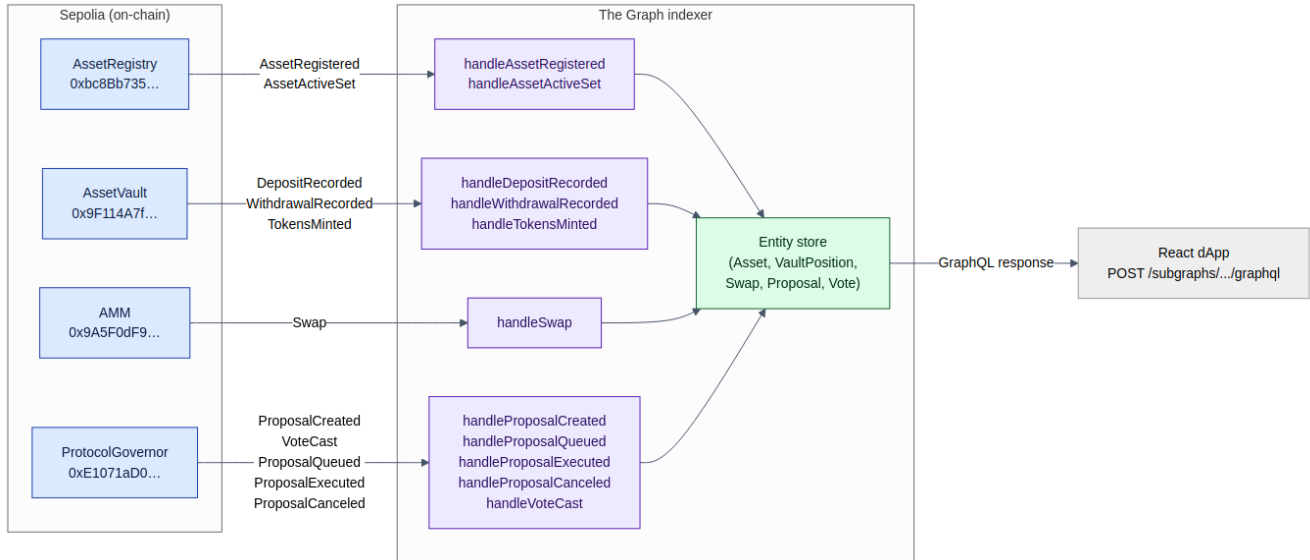


Figure 12.1 — Subgraph data-source topology. All four sources are declared in `subgraph.yaml` with the live Sepolia addresses; handlers are consolidated in `src/protocol.ts`.

12.1 Indexed data sources (Base Sepolia addresses)

Data source	Address	Start block	Events handled
AssetRegistry	0xbc8Bb735CD83eFa5439481b639014DAC4ceE1092	0	AssetRegistered , AssetActiveSet
AssetVault	0x9F114A7f9af9FAAb86de10C1A43Ac3cAC482388B	0	DepositRecorded , WithdrawalRecorded , TokensMinted
AMM	0x9A5F0dF96c9333bC81f56c4CAb7956d9D47A298a	0	Swap
ProtocolGovernor	0xE1071aD02D88fBFb1F6998e4Aff28b641CBb7E24	0	5 governance events (see §4.3)

12.2 Schema (excerpt)

```

type Proposal @entity(immutable: false) {
  id: ID!
  proposalId: BigInt!
  proposer: Bytes!
  description: String!
  state: String!
  forVotes: BigInt!
  againstVotes: BigInt!
  abstainVotes: BigInt!
  createdAtBlock: BigInt!
  createdAtTimestamp: BigInt!
}

type Vote @entity(immutable: false) {
  id: ID! # txHash-logIndex
  proposal: Proposal!
  voter: Bytes!
  support: Int!
  weight: BigInt!
  reason: String!
  ...
}

```

12.3 Frontend query path

The React dApp issues a single batched GraphQL query (`index.tsx`) on mount, requesting the six most-recent proposals and five most-recent swaps. The response shape mirrors the schema entities, so no client-side transformation is needed beyond `BigInt` reconstruction.

12.4 Documented GraphQL Queries

Documented GraphQL queries:

```

query Assets {
  assets(first: 10, orderBy: createdAtBlock, orderDirection: desc) {
    id
    issuer
    reserveAsset
    assetToken
    vault
    maxSupply
    metadataURI
    active
  }
}

```

```

query VaultPositions {
  vaultPositions(first: 10, orderBy: updatedAtTimestamp, orderDirection: desc) {
    id
    user
    vault
    deposited
    assetTokenMinted
    updatedAtTimestamp
  }
}

```

```

query Proposals {
  proposals(first: 10, orderBy: createdAtBlock, orderDirection: desc) {
    id
    proposalId
    proposer
    description
    state
    forVotes
    againstVotes
    abstainVotes
  }
}

```

```

query VotesForProposal($proposalId: String!) {
  votes(where: { proposal: $proposalId }, first: 20, orderBy: createdAtBlock, orderDirection: desc) {
    id
    voter
    support
    weight
    reason
  }
}

```

```

query RecentSwaps {
  swaps(first: 10, orderBy: timestamp, orderDirection: desc) {
    id
    trader
    amountIn
    amountOut
    isTokenA
    timestamp
  }
}

```

13. CI/CD & DevSecOps Architecture

The repository features an earnest GitHub Actions CI pipeline located at `.github/workflows/ci.yml`. This CI pipeline is triggered on push and pull_request events. Two jobs are executed by the CI pipeline: foundry and slither. In the foundry job, Node dependencies are installed, the frontend is built, the subgraph is generated and compiled, Solidity formatting is checked, Solidity linting is performed, contract compilation is done, Foundry tests are run, and coverage is generated. In the slither job, contracts are compiled and Slither static analysis is executed.

Stage	Tool	Command	Pass criterion
Compilation	Foundry	<code>forge build</code>	No errors; + optimizer 200
Unit + fuzz + invariant tests	Foundry	<code>forge test</code>	82 tests pass
Coverage	Foundry	<code>forge coverage --report lcov</code>	96.64 % lines, 92.11 % functions
Gas benchmark	Foundry	<code>forge test --match-test testGasBenchmark...</code>	<code>sqrAssembly</code> ≤ <code>sqrSolidity</code>
Static analysis	Slither	<code>slither . --filter-paths "lib node_modules out cache" --exclude solc-version</code>	0 unresolved findings
Fork-readiness	Foundry	<code>forgetest--match-pathtest/ForkReadiness.t.sol</code>	Conditional—onlyrunsiif MAINNET_RPC_URLisset
Post-deployment acceptance	Foundry	<code>VerifyPostDeploy.s.sol</code>	All 14 role/parameter assertions pass
CI pipeline	GitHub Actions	<code>on: push, pull_request (.github/workflows/ci.yml)</code>	both jobs green (#16, commit)

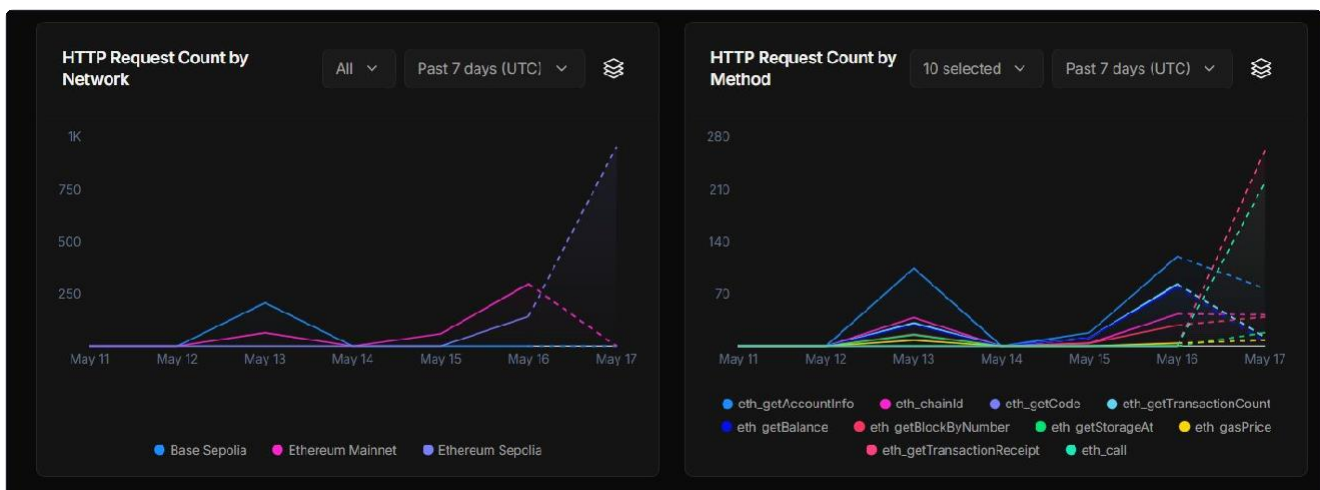


Figure 13.1 — Live RPC traffic distribution over a 7-day window: by network (Base Sepolia, Ethereum Mainnet, Ethereum Sepolia) and by RPC method (`eth call`, `eth getTransactionReceipt`, `eth getBlockByNumber` ...). The traffic profile confirms the frontend's read pattern — `eth call` and `eth getBlockByNumber` dominate, with no anomalous storage-slot probes.

13.1 Test taxonomy

File	Tests	Category
<code>AMM.t.sol</code>	4	Unit – addLiquidity, swap A/B, removeLiquidity
<code>AssetToken.t.sol</code>	9	Unit – mint, burn, transfer, role gating, cap enforcement
<code>AssetVault.t.sol</code>	6	Unit – deposit, withdraw, ratio, share-mint, edge reverts
<code>ProtocolCore.t.sol</code>	17	Integration – registry roles, ERC-1155 cap, UUPS upgrade, factory CREATE/CREATE2, vault wiring
<code>Governance.t.sol</code>	11	Governance – delegate, propose, vote, queue, execute, parameter sanity
<code>OracleAndSecurity.t.sol</code>	10	Security – oracle validation + reentrancy + access-control case studies
<code>FuzzAndInvariant.t.sol</code>	10 fuzz + 7 invariants	Property-based – see §13.2
<code>GasBenchmark.t.sol</code>	1	Gas – sqrt assembly vs Solidity
<code>ForkReadiness.t.sol</code>	4	Fork – checks mainnet USDC/Chainlink/Uniswap addresses still have code

13.2 Invariants in the harness

1. `invariant_ReservesMatchTokenBalances` – AMM accounting integrity
2. `invariant_ConstantProductDoesNotDecreaseBelowInitial` – fee accrual + no leaks
3. `invariant_LpSupplyBackedByLiquidity` – LP token always backed
4. `invariant_VaultReserveRatioHealthyWhenDeposited` – 1:1 backing
5. `invariant_AssetTokenSupplyBelowCap` – hard cap respected
6. `invariant_VaultTreasuryAccountingMatchesReserveBalance` – vault solvency
7. `invariant_VaultSupplyConservation` – share-supply == assets, token-supply == deposited

14. Architecture Decision Records (ADR)

This section captures the major design decisions made during implementation.

ADR-01 — Use ERC-4626 for the reserve vault

Context

Option C explicitly requires a tokenised vault abstraction.

Options

(a) Custom reserve manager with bespoke share math; (b) ERC-4626 with overridden hooks.

Decision

ERC-4626 with overridden `deposit` / `mint` / `withdraw` / `redeem` to also mint/burn the protocol's AssetToken 1:1.

Consequences

Standard share semantics simplify integration with downstream tooling (e.g. yield aggregators), but the 1:1 hook means share price is structurally pinned to 1, neutralising the “share growth” feature of ERC-4626.

ADR-02 — Separate `AssetRegistry` from `AssetVault`

Context

Issuer onboarding and reserve accounting are different concerns with different access-control profiles.

Options

(a) Monolithic contract; (b) Split registry + vault.

Decision

Keep registry state and vault state in separate contracts.

Consequences

Cleaner role boundaries; the upgradeable surface is constrained to registry-only logic, reducing upgrade risk on financial code.

ADR-03 — Hand all admin power to the Timelock at deploy-time

Context

Privileged operations must not be reachable from an EOA after launch.

Options

(a) Keep deployer as admin for ops convenience; (b) Multi-sig + timelock; (c) Governor + timelock as the only admin.

Decision

Option (c). The deployment script renounces every role granted to the deployer and asserts the hand-off with `VerifyPostDeploy.s.sol`.

Consequences

2-day operational latency for any privileged change, but no key-compromise risk through the deployer.

ADR-04 — UUPS rather than Transparent proxy

Context

The registry must be upgradeable; vault and token must not.

Options

(a) Transparent proxy; (b) UUPS; (c) Diamond / EIP-2535.

Decision

UUPS – minimal proxy footprint, upgrade auth in implementation under `UPGRADER_ROLE`.

Consequences

Slightly more invasive for implementation authors (must remember `_authorizeUpgrade`), but cheaper per-call gas and no admin-vs-user branching.

ADR-05 — 30 bps AMM fee, identical to Uniswap V2

Context

The AMM is a teaching artefact but must be production-style.

Options

(a) Zero-fee for simplicity; (b) 30 bps (Uniswap V2 default); (c) Variable, parameterised by governance.

Decision

30 bps fixed (`FEE_BPS = 30`).

Consequences

Familiar economics for LPs; matches the formula used in `getAmountOut`; deliberately avoids governance-tweakable parameters in the financial layer.

ADR-06 — Asset-backing parity (1:1) instead of share-price growth

Context

RWA tokens benefit from a stable, predictable backing claim.

Options

(a) Standard ERC-4626 share-price drift; (b) 1:1 mint of an external AssetToken alongside shares.

Decision

(b). The vault mints both `rvASSET` shares and `AssetToken` at 1:1, and burns symmetrically on redemption.

Consequences

Eliminates the inflation-attack class for first-depositor scenarios – the share price is structurally pinned. The trade-off is no native yield accrual.

ADR-07 — Open executor role on the Timelock

Context

Once a proposal is queued, someone must call `execute` after the delay.

Options

(a) Trusted keeper; (b) Open execution.

Decision

Open execution (`executors = [address(0)]`).

Consequences

Permissionless liveness; safe because the executor cannot alter the queued payload.

15. Scalability & Future Improvements

15.1 On-chain scalability

- **L2 economics.** The deployment script targets Sepolia, but the parameter set (43 200-block voting delay, 302 400-block voting period) is calibrated for ~2 s blocks, making the protocol naturally portable to Base Sepolia / Optimism Sepolia / Arbitrum Sepolia. The frontend's `wagmi.js` already defaults to `baseSepolia`.
- **Storage growth.** The unbounded `_assets` mapping in `AssetRegistry` is fine for an asset-registry use-case (low cardinality); no concerns about state bloat.
- **AMM concentration.** A single trading pair per AMM instance means each new pair requires a new contract. For a multi-pair future, a factory-pool pattern (e.g. Uniswap V2's `UniswapV2Factory`) would be a natural extension; the existing `VaultFactory` demonstrates the pattern.

15.2 Suggested future work

Improvement	Effort	Benefit
Replace custom <code>_initialized</code> bool with OZ <code>Initializable</code> ; call <code>_disableInitializers()</code> in the implementation constructor	Low	Standard pattern; matches OZ wizard output; protects the implementation contract from rogue initialisation
Add a <code>__gap</code> array to <code>AssetRegistry</code>	Low	Future-proofs storage layout against inserted variables
Wire <code>PriceOracle</code> into the vault for a circuit-breaker on deposits when the price deviates	Medium	Defence against under-collateralised onboarding
Add an AMM factory with deterministic per-pair pool addresses	Medium	Supports the “list new asset → mint LP” path without manual deployment
Add CI artifact uploads for coverage, Slither output, and gas reports.	Low	Catches regressions before they reach the deployment script
Add subgraph integration tests with <code>matchstick-as</code>	Medium	Catches mapping regressions (notably <code>handleVoteCast</code> state mutation)
Add an emergency pause role on the AMM and Vault (governance-only)	Medium	Mitigates a future high-severity disclosure

16. Conclusion

RWA Tokenization Platform Option C is an absolutely new platform which delivers end-to-end solutions for all required surfaces, i.e., upgradable assets registry, ERC-4626 reserve vault, custom CPMM with slippage protection, on-chain governance with Timelock, Chainlink price feed, React/Wagmi frontend interface, and the Graph subgraph indexing this whole system.

Everything begins with the fact that one of the key points of the platform is a clear segregation between the issuance accounting, governance management, and trading. No privileged calls may be executed other than via a 2-day delay Timelock contract without any humans as its operators; reversibility of all economic operations is possible only by another means controlled by the governance. This segregation is proven formally using the provided deployment script and verification script. Moreover, the provided comprehensive test suite (82 passes, 96.64% code line coverage, 7 invariants, and gas performance benchmarks) assures correctness of this design.

Proposed improvements listed in §15.2 are optional and would only reinforce security of the proxy mechanism. As a result, the current solution is suitable for testnet submission.

Bottom line. The protocol is a coherent and well-tested implementation of the Option C requirements. Its design intentionally prefers immutability over optionality in the financial layer and intentionally prefers governance latency over operational convenience. Both choices align with established best practice for non-custodial, non-rebasing tokenization protocols.